Cats vs Dogs: Image Classification with CNNs

Computer Vision Final Project · Yashwanth · November 2026

1. Problem statement and real-world use

The task is to decide whether a photo shows a **cat** or a **dog**. It's a small binary problem, but it contains the core difficulties of real image classification: large variation in pose, lighting, background and image quality, occlusion, and noisy labels. The same pipeline (clean data, a CNN or transfer-learned backbone, honest held-out evaluation, explanation) underlies **retail product tagging** (auto-labelling catalogue photos), **healthcare imaging** (flagging scans for review, where explainability matters as much as accuracy), and **security** (detecting people or vehicles in camera feeds). We compare a CNN trained from scratch, a regularized CNN and MobileNetV2 transfer learning, explain predictions with Grad-CAM, and deploy the best model in a Streamlit app.

2. Dataset

Microsoft PetImages, the Kaggle Cats vs Dogs set (https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset, CDLA-Permissive-2.0): 25,000 photos, 12,500 per class, in *Cat/* and *Dog/* folders. **Cleaning:** every file was fully decoded with Pillow and with TensorFlow's JPEG decoder (the one used in training). **233 of 25,002 files were dropped**: 173 BMP, 51 GIF, 4 PNG, 1 PSD, 4 unreadable. Most were other image formats saved with a .jpg name, and two were Windows Thumbs.db files. Images vary from 50 px to 500 px (median 448×375), so all are resized to a square input. **Split:** stratified 70/15/15 with seed 42, giving 17,339 / 3,715 / 3,715 images, balanced within 0.2%. The test split was used only once, for the final evaluation.

3. Methods

Preprocessing is a layer inside each model: rescaling to [0, 1] for the CNNs and [-1, 1] for MobileNetV2. Training, evaluation and the app therefore all feed raw RGB pixels. **Augmentation** (training batches only): random horizontal flip, rotation (±29°), zoom (±15%) and brightness (±15%). All models use Adam, binary cross-entropy, and EarlyStopping / ModelCheckpoint / ReduceLROnPlateau on validation loss.

	Baseline CNN	Improved CNN	MobileNetV2
Input	128×128	128×128	160×160
Body	3×[Conv-ReLU-MaxPool] 32/64/128 filters	same + BatchNorm, Dropout 0.1 per block	ImageNet MobileNetV2 (frozen, then top 30 fine-tuned)
Head	Flatten-Dense128-Dropout0.5-sigmoid	same + BatchNorm on Dense	GAP-Dropout0.2-sigmoid
Augmentation	no	yes	yes
Learning rate	1e-3	3e-4	1e-3, then 1e-5

Getting the improved CNN right took three runs. Run 1 (BatchNorm + Dropout 0.2 per block, lr 1e-3) *underfit* and scored below the baseline. Diagnosis showed the BatchNorm-scaled 32,768 flattened features blew up the 4.2M-weight Dense layer (its ReLUs died and the model stuck at 50%), on both Keras backends. Adding BatchNorm after the Dense layer, BN momentum 0.9 and lr 3e-4 fixed it. Run 3 then relaxed the LR scheduler, which in run 2 had halved the rate five times on the noisy validation curve. **Hardware:** TensorFlow has no native-Windows GPU support, so the GPU models trained with Keras 3 on the PyTorch backend (RTX 5060 Laptop GPU). The baseline trained on CPU. The saved models reload under TensorFlow with identical test metrics. **Deployment:** a Streamlit app shows the label, the confidence and a Grad-CAM overlay. Because a binary model has to say cat or dog even for a car, a stock ImageNet MobileNetV2 first checks that one of its top-5 classes is a cat or dog breed.

4. Results

Model	Test acc.	Precision	Recall	F1	ROC-AUC	Params	Epochs	Train time
Baseline CNN	84.6%	0.836	0.859	0.848	0.919	4.29M	10	23.6 min CPU
Improved CNN	85.7%	0.855	0.860	0.858	0.934	4.29M	26	21.6 min GPU
MobileNetV2	**98.9%**	**0.988**	**0.990**	**0.989**	**0.999**	**2.26M**	**17**	**23.7 min GPU**

Table 1. Test-set results (3,715 images; dog = positive class).

MobileNetV2 is clearly best with 98.9% accuracy and AUC 0.999, using half the parameters of the scratch CNNs. Its ImageNet features reach about 98% validation accuracy after a single epoch, and fine-tuning adds the rest. The **baseline overfits**: training accuracy rises to 97.4% while validation stalls at about 85% (Fig. 1). The **improved CNN** keeps train and validation within a few points and has lower validation loss (0.306 vs 0.351). Regularization fixed the overfitting, but test accuracy rose only from 84.6% to 85.7%.

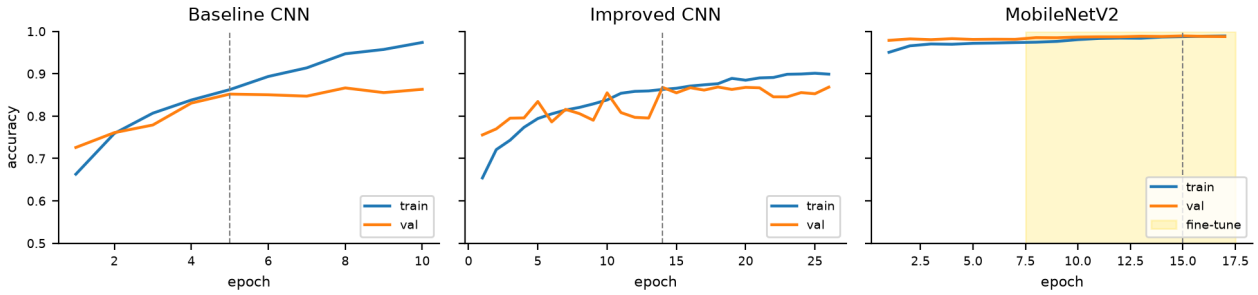


Figure 1. Train vs validation accuracy (dashed line: best epoch, kept by checkpointing; shaded: MobileNetV2 fine-tuning).

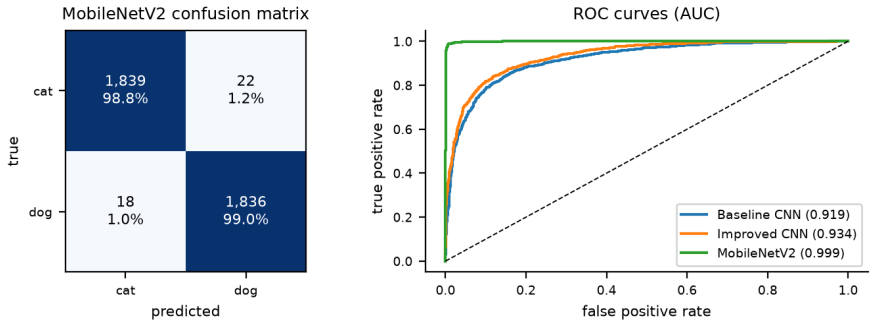


Figure 2. MobileNetV2 confusion matrix (row %) and ROC curves for all three models.

Error analysis. The most confidently wrong test images fall into clear groups. (1) *Both species or label noise:* #277 (labelled cat) shows a dog next to a cat, and #2082 (labelled dog) is a town logo with no animal at all. (2) *Puppies and curled-up poses* look cat-like. (3) *Cage bars and occlusion.* (4) *Black fur in harsh light, face hidden.* (5) *A small animal in a busy scene with people.* **Grad-CAM** (Fig. 3) shows MobileNetV2 attending to the animal, even on its mistakes. On #277 it highlights the dog. The scratch CNN often focuses on **background shortcuts** such as cage bars, picture frames and carpet, which explains its brittleness and is the strongest argument for transfer learning on a dataset of this size.



Figure 3. Grad-CAM (red = most influential). Same cat photo through both models, then two MobileNetV2 mistakes.

5. Limitations

- **Binary task only.** Any input is forced into cat or dog. The ImageNet guard catches obvious non-pets (on three wallpaper photos both models gave 61-92% confident answers, and the guard flagged all three), but it depends on ImageNet's classes.
- **Dataset bias.** Mostly indoor pet photos from one adoption website. Poses, cages and backgrounds correlate with labels, and the scratch CNNs exploit this. A few images are mislabelled, so the measured error is slightly pessimistic.
- **Single split, single seed.** No cross-validation or confidence intervals, so small gaps such as baseline vs improved CNN (+1.2 points) are not conclusive.

6. Future work

- Multi-class **breed** classification (Oxford-IIIT Pet, 37 classes) and a learned **"neither"** class trained on real negative images instead of an external guard.
- **TFLite / ONNX export** of MobileNetV2 for on-device inference, comparing size and latency.
- Clean the mislabelled images, add test-time augmentation, and report multi-seed confidence intervals.